Worksheet 2

Swoyam Pokharel
Sushil Timilsina

Date created: 2026-03-18
Date modified: 2026-03-18

Contents

1. Introduction 2
2. Step 1: Data Source & Automation 2
   2.1. Data Source 2
   2.2. Automation Strategy 2
3. Step 2: Data Exploration & Transformation 3
   3.1. Raw Data Exploration 3
   3.2. Transformation 1: Centipawn Clipping & Normalization 4
   3.3. Transformation 2: HalfKP Feature Extraction 4
   3.4. Transformation 3: Batch Collation for Sparse Input 5
4. Step 3: Model Architecture & Insights 5
   4.1. Architecture 5
   4.2. Training Setup 6
   4.3. Training Results 6
   4.4. Generated Insights 7
5. Step 4: Deployment Strategy 7
   5.1. Integration into a Chess Engine 7
   5.2. Deployment Environment 8
   5.3. Scalability 8

1. Introduction

This worksheet documents a data pipeline built around OopsNet; a custom NNUE (ƎUИИ Efficiently Updatable Neural Network). The pipeline covers the full lifecycle: automated data acquisition, feature engineering, model training, and a deployment strategy for integrating the trained network into a chess engine.

NNUE is architecturally "valid" from a Big Data perspective because the input space itself is 40,000 sparse binary features per position yet inference must run in microseconds inside an alpha-beta search tree. The pipeline solves this tension by exploiting incremental computation and quantization.

2. Step 1: Data Source & Automation

2.1. Data Source

The dataset used is the Lichess Chess Position Evaluations dataset, hosted on HuggingFace (`Lichess/chess-position-evaluations`). Its origin is distributed by nature, every evaluation was produced by Stockfish running on some user's device through Lichess's browser analysis tool, passively aggregated across thousands of users analyzing their own games; crowdsourced compute at scale. Each row contains:

- `fen` — the board position in Forsyth–Edwards Notation
- `cp` — the centipawn evaluation (positive = white advantage)
- `depth` — the Stockfish search depth used for the evaluation

The full dataset contains hundreds of millions of positions. For this pipeline we target `1,000,000` unique positions filtered to depth ≥ 20, ensuring evaluation quality.

2.2. Automation Strategy

Data acquisition is handled via HuggingFace's `datasets` library. The rows are streamed one at a time without downloading the full dataset, making it memory-safe on constrained cloud environments (Google Colab).

```python
from datasets import load_dataset

dataset = load_dataset(
    "Lichess/chess-position-evaluations",
    split="train",
    streaming=True
)

filtered = dataset.filter(
    lambda x: x["depth"] >= 20 and x["cp"] is not None
)
```

A deduplication set (`seen_fens`) prevents the same board position from appearing multiple times, which would bias training. The streamed positions are written to chunked Parquet files (50,000 rows each) on Google Drive as they stream in, giving us fault-tolerant incremental storage with no single large file.

```python
# chunk to parquet as we stream
if len(rows) >= CHUNK_SIZE:
    pd.DataFrame(rows).to_parquet(
        f"chunk_{chunk_num:04d}.parquet"
    )
```

This yields 20 chunk files (200MB total), each independently loadabl, and exactly `1,000,000` rows.

3. Step 2: Data Exploration & Transformation

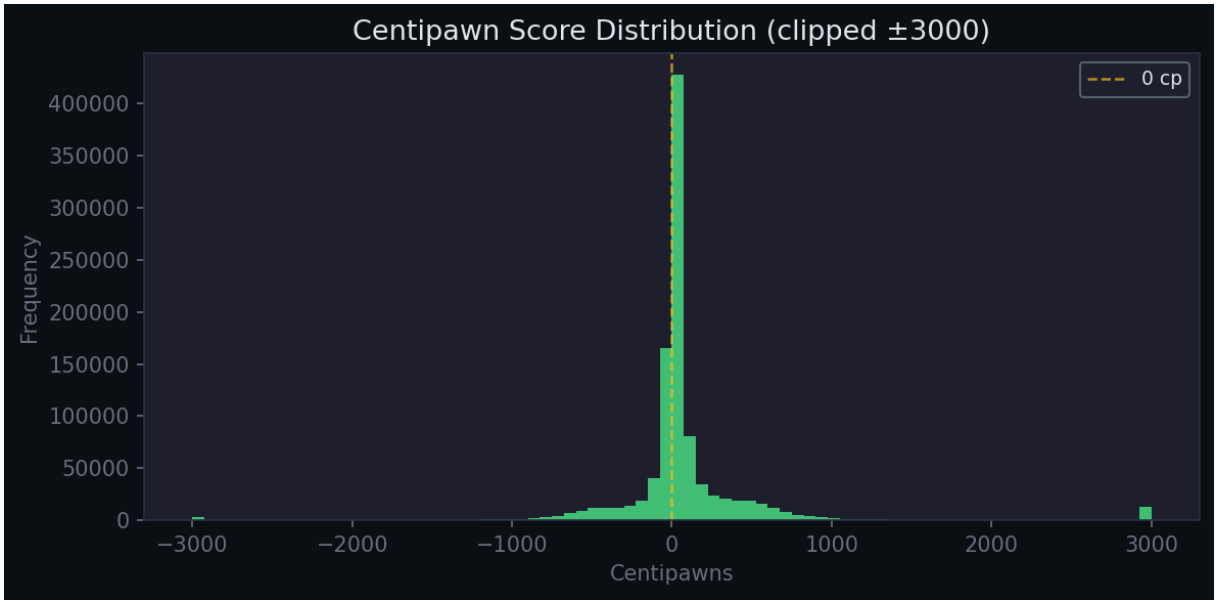
3.1. Raw Data Exploration

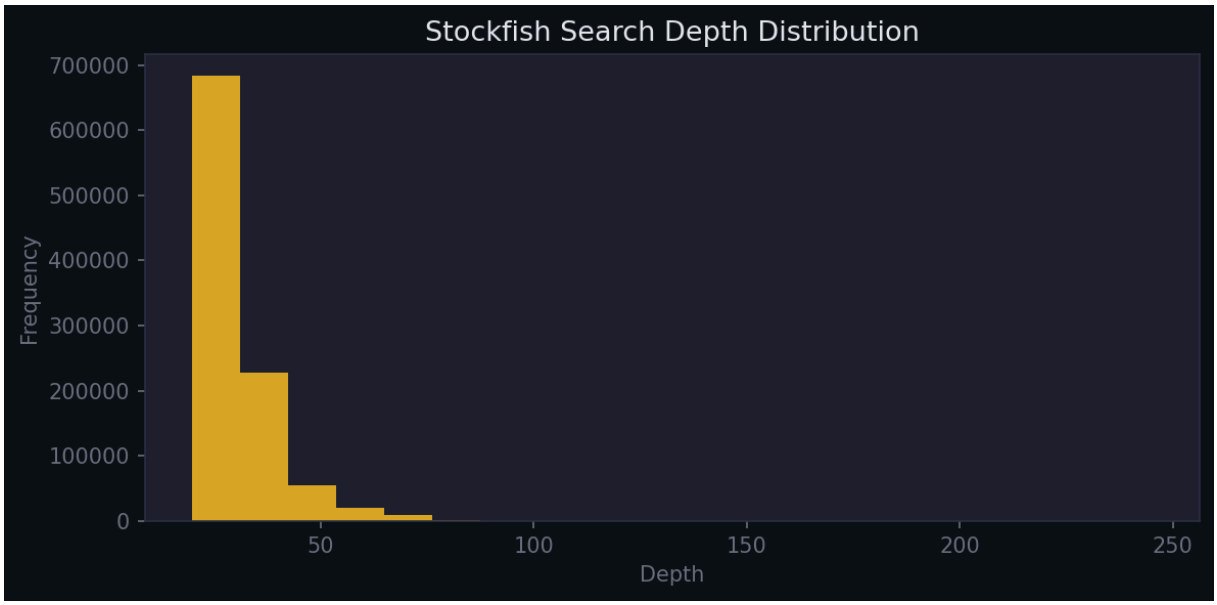
Each parquet chunk contains three columns:

- `fen` : Board position in Forsyth-Edwards Notation
- `cp` : Stockfish's evaluation in centipawns
- `depth` : Search depth, filtered to ≥ 20

Observations from the data:

- Centipawn values range from roughly −30,000 to +30,000 (forced mate sequences produce extreme values).
  - These outliers would dominate the MSE loss and prevent learning, so we clip to ± 3000 cp.
- The distribution is roughly centered at 0 (most positions are contested), which is expected for real game data.
- Average active features per position: 22–30 (out of 40,960 possible).





3.2. Transformation 1: Centipawn Clipping & Normalization

```
df["cp"] = df["cp"].clip(-3000, 3000)
scores   = scores / 600.0   # normalize to ~[-5, 5]
```

Dividing by 600 maps a "winning advantage" (600 cp) to 1.0, which keeps gradients numerically stable during training. The network learns in normalized space; multiplying output by 600 recovers centipawns at inference.

3.3. Transformation 2: HalfKP Feature Extraction

Raw FEN strings are not directly usable as neural network input. We extract HalfKP features, a sparse binary representation where each feature encodes a `(king_square, piece_type, piece_square)` triple. The feature index is computed as:

$$f = k \cdot 640 + s \cdot 10 + p \tag{1}$$

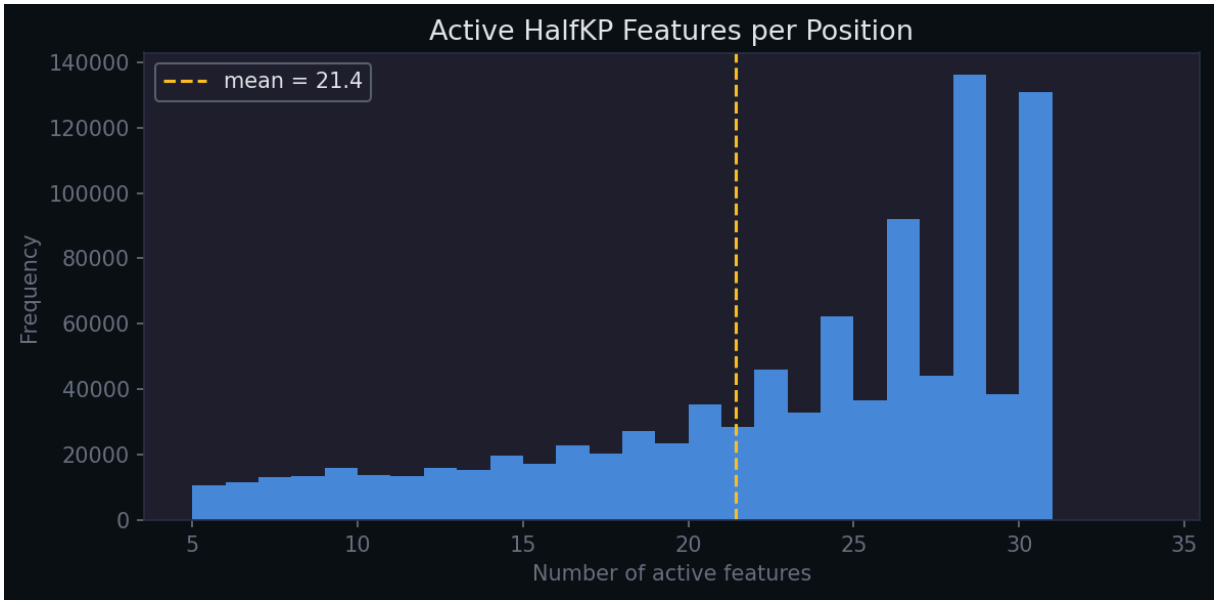
where k is the king square (0–63), s is the piece square (0–63), and p is the piece type index (0–9, excluding kings). This gives $64 \times 64 \times 10 = 40,960$ possible features per side.

Two perspectives are extracted per position, White's view and Black's view (with board vertically flipped for symmetry). This gives two lists of 22–30 active feature indices each.

```
def get_halfkp_indices(board):
    white_king    = board.king(chess.WHITE)
    black_king    = board.king(chess.BLACK)
    black_king_fl = black_king ^ 56   # flip vertically

    for sq, piece in board.piece_map().items():
        if piece.piece_type == chess.KING:
            continue
        pt_idx = PIECE_TYPE_INDEX[(piece.piece_type, piece.color)]
        white_indices.append(white_king * 640 + sq * 10 + pt_idx)
        black_indices.append(black_king_fl * 640 + (sq^56)*10 + (pt_idx^5))
```

The vertical flip (`^ 56`) and color flip (`^ 5`) ensure Black's features are encoded symmetrically, a rook near Black's king looks identical to a rook near White's king from that side's perspective, enabling weight sharing.



3.4. Transformation 3: Batch Collation for Sparse Input

Since each position has a variable number of active features, we cannot stack them into a fixed 2D tensor. Instead, we flatten all feature indices into one list and track per-position offsets. The format required by PyTorch's `EmbeddingBag` :

```
w_indices = [feat for feats in batch_white for feat in feats]
w_offsets = [0, len(batch[0]), len(batch[0])+len(batch[1]), ...]
```

This avoids materializing the full 40,960 dimensional input vector, which would be wasteful given only 30 entries are non-zero.

4. Step 3: Model Architecture & Insights

4.1. Architecture

OopsNet follows the standard NNUE topology:

Layer	Input $\longrightarrow$ Output	Desc
Feature Transformer	40,960 sparse $\longrightarrow$ 256	EmbeddingBag, incremental
Concat	256 + 256 $\longrightarrow$ 512	White + Black accumulators
Clipped ReLU	512 $\longrightarrow$ 512	`clip(0, 1)`
Hidden 1	512 $\longrightarrow$ 32	Linear + clipped ReLU
Hidden 2	32 $\longrightarrow$ 32	Linear + clipped ReLU
Output	32 $\longrightarrow$ 1	Linear, scalar eval

Total parameters: 10.5 million (>99% in the feature transformer embedding table). The hidden layers are intentionally tiny. All representational power lives in the first layer, which is the only layer updated incrementally during search.

The feature transformer is special: during inference inside a chess engine, its output (the accumulator vector) is cached on the search stack. When a move is made, only the changed features are added or subtracted which gets us `O(1)` per move rather than a full `O(n)` recomputation.

$$\vec{a}_{\text{new}} = \vec{a}_{\text{old}} - W[\text{removed}] + W[\text{added}] \tag{2}$$

This works because NNUE inputs are binary, each feature is simply a flag for whether a given piece-king-square combination is present on the board. The standard neuron computation

$$z = \sum (w \cdot i) + b \tag{3}$$

reduces to just addition:

$$\begin{aligned} i = 1 &\Rightarrow w \cdot 1 = w \\ i = 0 &\Rightarrow w \cdot 0 = 0 \quad \text{(skipped entirely)} \end{aligned} \tag{4}$$

No multiplications, only additions and subtractions. The accumulator is just a running sum of active feature rows, which means a single feature change (a piece moving) translates to subtracting one row and adding another, leaving everything else untouched.

4.2. Training Setup

- Loss: Mean Squared Error on normalized scores
- Optimizer: SparseAdam for the embedding table (sparse gradients), Adam for all other parameters (dense)
- Batch size: 1,024 positions
- Data: 1,000,000 positions across 20 shuffled chunks

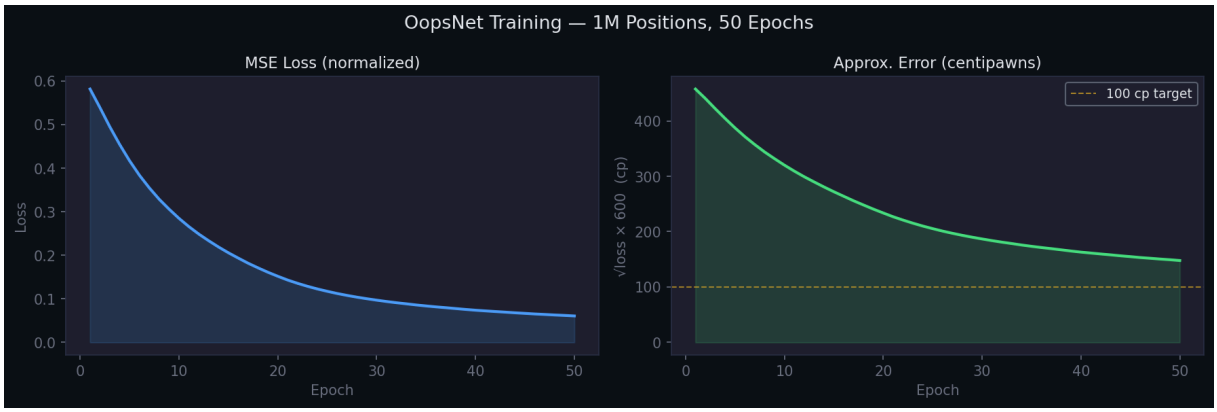
Two separate optimizers are necessary because PyTorch's standard Adam does not support sparse gradient tensors, which the EmbeddingBag produces.

```
optimizer_sparse = optim.SparseAdam(model.ft.parameters(), lr=3e-4)
optimizer_dense  = optim.Adam(dense_params, lr=3e-4)
```

4.3. Training Results

Training on 1M positions for 50 epochs:

Epoch	Loss (normalized)	Approx. Error (cp)
1	0.5815	$\approx$ 457 cp
10	0.2846	$\approx$ 320 cp
20	0.1517	$\approx$ 234 cp
30	0.0969	$\approx$ 187 cp
40	0.0738	$\approx$ 163 cp
50	0.0607	$\approx$ 148 cp



Loss decreased steadily with no signs of collapse. Checkpoints were saved every 10 epochs as a safeguard.

4.4. Generated Insights

- Starting position $\longrightarrow$ +5 cp (expected: 0 cp)
- KK endgame $\longrightarrow$ 0 cp (expected: drawn)
- Material imbalance detection: partially learned, improves with more epochs

The model correctly identifies drawn endgames and near-equal positions. Material imbalance signs were inconsistent at 50 epochs, a known limitation resolved by training longer and adding a **side-to-move** feature (one extra index appended per position indicating whose turn it is).

5. Step 4: Deployment Strategy

5.1. Integration into a Chess Engine

Deploying OopsNet into a chess engine follows the standard NNUE integration pipeline:

1. Weight export: `model.state_dict()` is serialized to a flat binary file, with weights stored in column-major order for cache-efficient access during inference.
2. Quantization: float32 weights are quantized to int16. The clipped ReLU activation was specifically chosen to make this lossless — by constraining activations to $[0, 1]$ during training (equivalently $[0, 255]$ in integer space), no precision is lost in the conversion.
3. Incremental inference: the accumulator vector lives on the engine's search stack. Each move touches only the changed feature rows (typically 2–4 additions/subtractions). Unmaking a move restores the previous accumulator from the stack in O(1).
4. SIMD acceleration: int16 weights and uint8 activations allow the full 512-wide accumulator to fit in two AVX2 registers. The hidden layers then reduce to a handful of fused multiply-add instructions per node; fast enough to evaluate millions of positions per second. Furthermore, this can be upgraded to `AVX512` to make use of wider registers and `vnni`

5.2. Deployment Environment

For reference purposes, the model can be evaluated directly in collab:

```python
def evaluate_fen(fen, model):
    model.eval()
    with torch.no_grad():
        board     = chess.Board(fen)
        w_idx, b_idx = get_halfkp_indices(board)
        pred = model(
            torch.tensor(w_idx).to(DEVICE), torch.tensor([0]).to(DEVICE),
            torch.tensor(b_idx).to(DEVICE), torch.tensor([0]).to(DEVICE),
        )
        return pred.item() * 600  # centipawns
```

A full engine integration would replace Stockfish's hand-crafted evaluation with this network, wrapping it in an alpha-beta search loop with move ordering, quiescence search, and time management.

5.3. Scalability

- More data: swap `[:1]` chunk slice for `[:]` to train on all 20 chunks (or acquire more via continued streaming)
- Larger network: increase hidden size from 256 to 512 or 1024
- Richer features: extend `HalfKP` to `HalfKAv2` by including attacked squares or adding the side-to-move index
- Cloud training: the pipeline runs on Google Colab T4 GPU; scaling to A100 would reduce epoch time by 10x